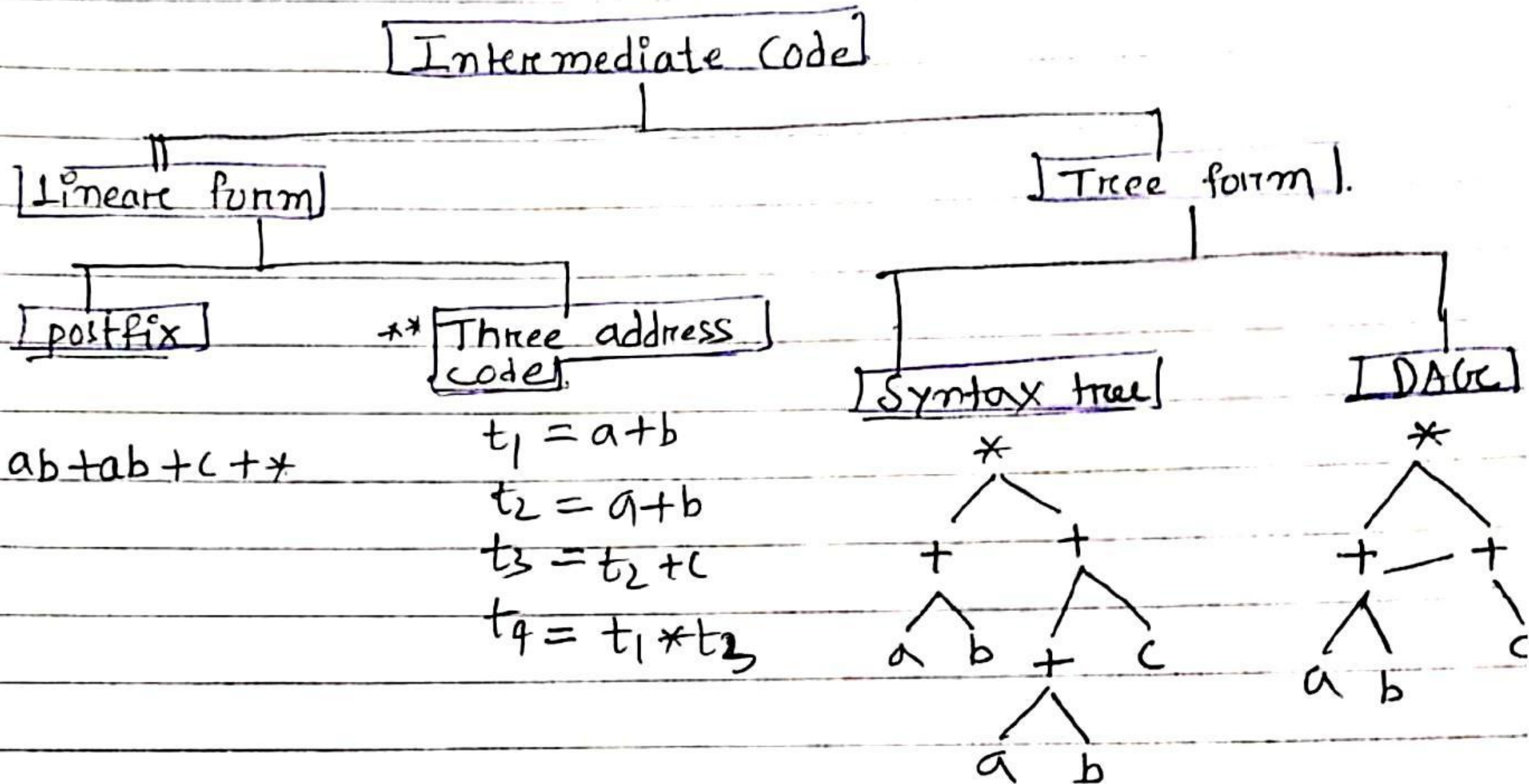


INTERMEDIATE CODE GENERATION

81

Intermediate Code generation =



Types of 3 address Code:

- ① $x = y \text{ of } z$
- ② $x = op\ y$
- ③ $x = y$
- ④ if $x \text{ (relation } op) y$ goto L.
- ⑤ goto L.
- ⑥ $A[i] = x$
 $y = A[i]$.
- ⑦ $x = *p$
 $y = \&y$.

• Various way to represent three address code =

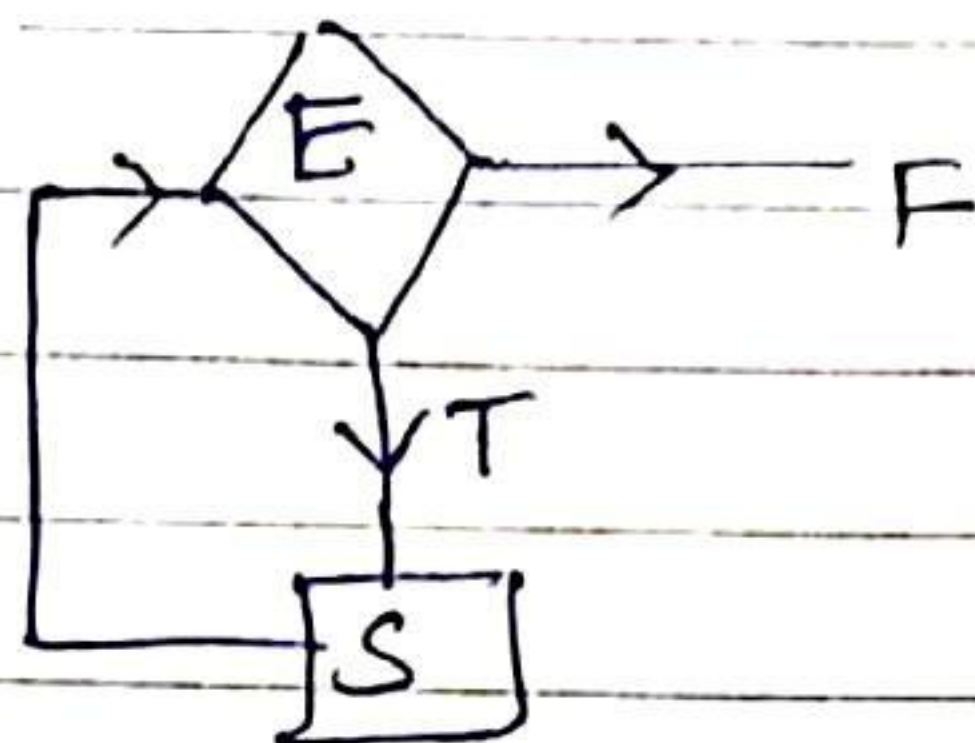
given instruction: $(a+b) * (c+d) + (a+b+c)$

- 1) $t_1 = a + b$
- 2) $t_2 = a + b - t_1$
- 3) $t_3 = c + d$
- 4) $t_4 = t_2 * t_3$
- 5) $t_5 = a + b$
- 6) $t_6 = t_5 + c$
- 7) $t_7 = t_4 + t_6$

3 ways =

Quadruples				TRIPLE			Indirect triples.
Opr	op1	op2	result	Opr	op1	op2	
1) +	a	b	t ₁	1) +	a	b	1) (i) (1)
2) -	t ₁	-	t ₂	2) -	t ₁		2) (ii) (2)
3) +	c	d	t ₃	3) +	c	d	3) (iii) (3)
4) *	t ₂	t ₃	t ₄	4) *	t ₂	t ₃	4) (iv) (4)
5) +	a	b	t ₅	5) +	a	b	5) (v) (5)
6) +	t ₅	c	t ₆	6) +	t ₅	c	6) (vi) (6)
7) +	t ₄	t ₆	t ₇	7) +	t ₄	t ₆	7) (vii) (7)
adv: Statement can be moved around.				adv: space is not wasted.			adv: Statements can be moved.
dis: too much of space wasted.				dis: statements cannot be moved.			dis: Two memory access.

ex = While E do S. (how to convert while loop into three address code)



$L: \text{if } (E == 0) \text{ goto } L_1$ S $\text{goto } L$ $L_1:$

→ When E is false goto L_1 , otherwise S .

OR

$L: \text{if } (E) \text{ goto } L_1$ goto last $L_1: S$ $\text{goto } L$ last:

ex

while $(a < b)$ do $x = y + z$

$L: \text{if } (a < b) \text{ goto } L_1$ goto last $L_1: t = y + z$ $x = t$ $\text{goto } L$ last:
--

• Back patching :

→ Whenever a statement is not in the tree add code, then by using all 'f' statements make them into tree address code.

ex =

$\boxed{\text{if}(a < b) \text{ then } t=1 \text{ else } t=0.}$ → not in the form of three address code.

(i): $\text{if}(a < b) \text{ goto } (i+3)$

(i+1): $t=0$

(i+2): $\text{goto } (i+4)$

(i+3): $t=1$

(i+4)

→ now in the form of three address code.

ex =

$\boxed{\text{if}(\overset{t_1}{(a < b)} \text{ and } \overset{t_2}{(c < d)} \text{ or } \overset{t_3}{(e < f)}) \text{ then } t_4 = 1 \text{ else } 0}$

100) $\text{if}(a < b) \text{ goto } 103$

101) $t_1 = 0$

102) $\text{goto } 104$

103) $t_1 = 1$

104) $\text{if}(c < d) \text{ goto } 107$

105) $t_2 = 0$

106) $\text{goto } 108$

107) $t_2 = 1$

108) $\text{if}(e < f) \text{ goto } 111$

109) $t_3 = 0$

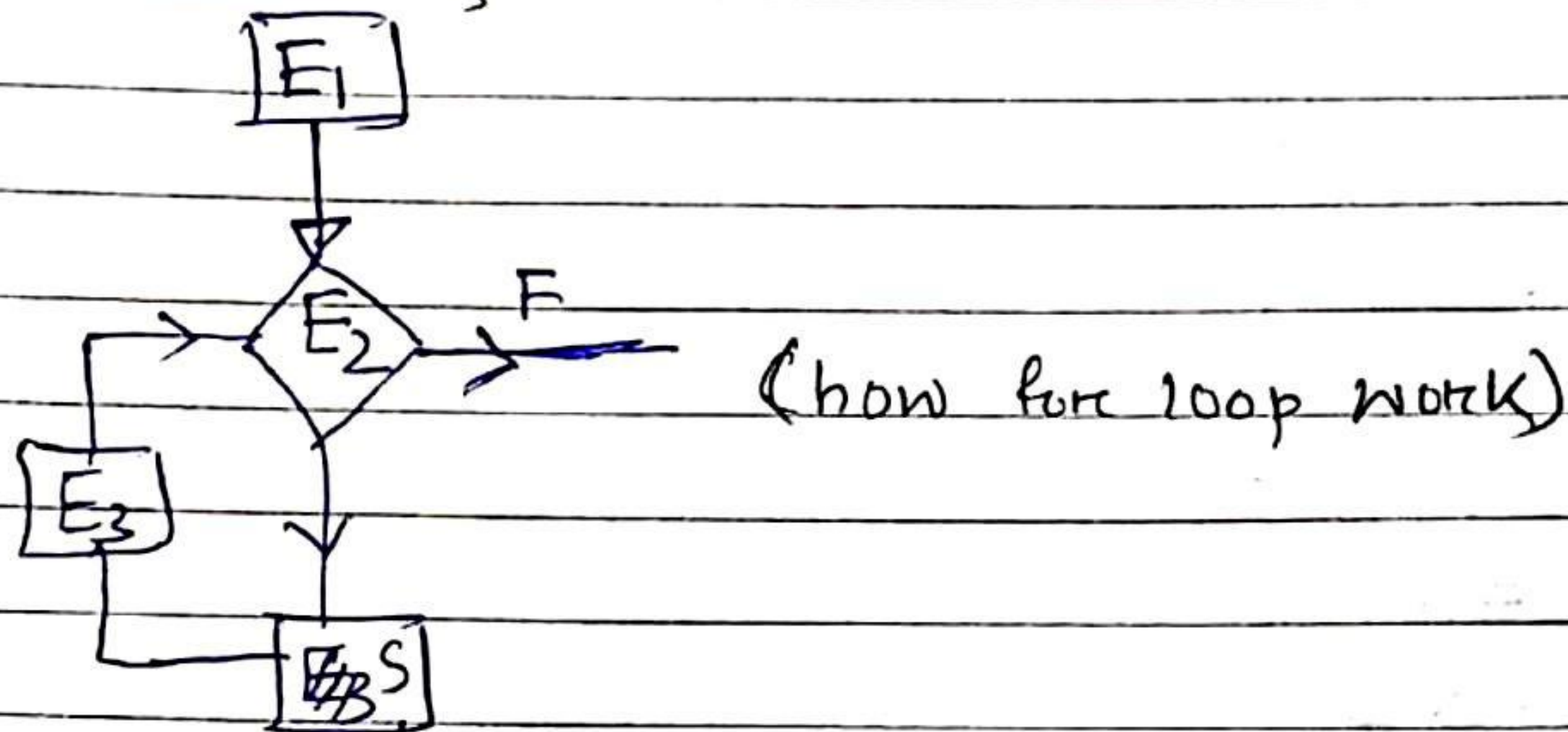
110) $\text{goto } 112$

111) $t_3 = 1$

112) $t_4 = t_1 \text{ and } t_2$

113) $t_5 = t_4 \text{ or } t_3$ ✓

ex = $\text{for } (E_1; E_2; E_3) \text{ (for loop to 3-add code)}$
 $\{ S; \}$



for loop represent in the form of three address code =

$P(i=0; i < 10; i++)$ $a = b + c;$

$i = 0$

L: if ($i < 10$) goto L1
 goto Last

L1: $t = b + c$

$a = t$

$t = i + 1$

$i = t$

goto L

Last:

ex = $\text{switch } (i+j) \text{ (switch case to 3-add code)}$

$\{$

case(I): $a = b + c;$

break

case(II): $p = q + r$

break;

default: $n = y + z;$

} break;

Switch Statement convert into three add code =

```

t = i + j
goto test
L1: t1 = b + c
    a = t1
    goto Last
L2: t2 = a + R
    p = t2
    goto Last
L3: t3 = y + z
    u = t3
    goto Last.
test: if (t == 1) goto L1
      if (t == 2) goto L2
      goto L3
Last:

```

[Ex] = Two Dimensional array convert into 3-code add
code =

A[4][4]

00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

A[2 3]

$2 * 4 + 3 = 11$
 (no. of R) (E) (no. of C)
 after cross 11 element
 you will get output.

RMO:

00 01 02 03 10 11 12 13 20 21 22 23 ...
 CMO: 00 10 20 30 01 11 21 31 02 12 22 32 ...

RMO: $(R * \text{no. of element} + C) * \text{element size}$

$$x = A[y, z]$$

$A : 10 \times 20 \rightarrow$ ^{2D} array.

$$\text{Base add} + (y * 20 + z) * 4$$

(100)

(4 is element size in bit)

3-add code =

$$t_1 = y * 20$$

$$t_2 = t_1 + z$$

$$t_3 = t_2 * 4$$

$$t_4 = \text{base address of } A$$

$$x = t_4[t_3]. \quad (\text{Base Address add with offset})$$

after crossing this amount of number we get result of $A[y, z]$.